

```
0000 -----
compiler-- Copyright (C) 2024-2026 Vincent MORIN - Universit  de Bretagne Occidentale-- This program is free software: you can redistribute it and/or
or modify-- it under the terms of the GNU General Public License as published by-- the Free Software Foundation, either version 3 of the License, or
-- (at your option) any later version.-- This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without eve
n the implied warranty of-- MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the-- GNU General Public License for more details.-- You sh
ould have received a copy of the GNU General Public License-- along with this program. If not, see <https://www.gnu.org/licenses/>.--
-----
ngue mais-- dont le fonctionnement par changement de signe des r f rences de page dans la version d'origine-- de Peregrine Systems n'est pas tr s lo
ir.-- Lors de l'analyse syntaxique (PAR_PHASE) des pages virtuelles d'arbre sont allou es pour-- l'arbre DIANA.-- Dans la phase librairie
o l'on charge les arbres d'unit s "withes", des pages-- suppl mentaires-ultrieures sont allou es, pages "withes" qui n' sont pas sauvegard es--
0010 l' criture de l'unit  apr s analyse s mantique.-- La phase de v rification s mantique (SEM_PHASE) ajoute derri res les pages "withes" des pages-- qu
i devront, elles,  tre sauvegard es la phase WRITE_PHASE. Ainsi les pages "withes" coupent en deux-- la plage des pages sauvegard es-- Pages
AST | Pages withes | Pages SEM-- Une translation de recompaction avant sauvegarde est ainsi n cessaire et proc de en deux temps :-- On mar
que d'abord toutes les pages ne pas toucher dans la compaction (les pages au del -- de la derni re utilis e en SEM_PHASE) et toutes les pages d'unit s
withes-- On fait une recopie des pages de l'unit  sauvegard e afin de former une seule plage de-- pages sauvegard es dans le fichier de comp
ilation de l'unit --
-----
MORIN-Universit  de Bretagne Occidentale-janvier 2025-----
-----1-2-3-4-5-6-7-8-9-A-B-C-----with SEQUENTIAL_IO; separate (~IDL)-----procedure-----WRITE_LIB-----is D
ONT_MOVE:= array( VPG_IDX ) of BOOLEAN;-----INDIQUE SI UNE PAGE N EST-PAS L OBJET DE DEPLACEMENTS DE TRANSLATION-----NEW_UNIT_SEQ:= SEQ_TYPE;-----
-----LA NOUVELLE LISTE D UNITES APRES COMPACTION-----procedure-----MARK_DONT_MOVE_PAGES--( COMP_UNIT : TREE )-----begin
0020 K;-----is-----TRANS_WITH_SEQ:= SEQ_TYPE:= LIST( COMP_UNIT );-----LISTE-FERMETURE-TRANSITIVE DES WITH-----HIGH_BLOCK:= PAGE_IDX:= LAST_BLOC
K;-----begin-----TOUTES LES PAGES JUSQU A HIGH_BLOCK PEUVENT A PRIORI  TRE OBJET DE TRANSLATION DE COMPACTION-----LES AUTRES AU DESSUS N ONT PAS A
 TRE AFFECT ES-----DONT_MOVE( 1 .. HIGH_BLOCK ) := ( others=>FALSE );-----DONT_MOVE( HIGH_BLOCK + 1 .. MAX_VPG ) := ( others=>TRUE );-----PAR
MI LES-PAGES SOUS HIGH_BLOCK, LES PAGES WITHES N ONT PAS A  TRE AFFECT ES PAR DE LA RECOPIE DE COMPACTION-----MARK_TRANS_WITH:-----
declare-----TRANS_WITH:= TREE;-----begin-----while not IS_EMPTY( TRANS_WITH_SEQ ) loop-----POUR TOUTE LA LISTE TRANSITIVE DES UNITES W
ITHES-----POP( TRANS_WITH_SEQ, TRANS_WITH );-----SORTIR UN POINTEUR DE BLOC INFO D UNIT  WITH E (AVEC ENT TE SPECIALE NON HI)-----
-----MARK_DONT_MOVE_PAGES_WITHES:-----declare-----WITHED_UNIT:= TREE:= D( TW_COMP_UNIT, TRANS_WITH );-----POINTEUR VERS N
OEUD UNIT -WITH E-----FIRST_PAGE:= PAGE_IDX:= WITHED_UNIT.PG;-----NBR_PAGES_WITHED:= PAGE_IDX;-----PAGE_IDX( DI( XD_NBR_PAGES, WITHED_UNIT ) );-----be
gin-----NBR_PAGES_WITHED := PAGE_IDX( DI( XD_NBR_PAGES, WITHED_UNIT ) );-----for I in FIRST_PAGE .. FIRST_PAGE + NBR_PAGES_WITHED - 1 loop-----DONT_MOVE(
I ) := TRUE;-----LES PAGES DES UNITES WITHES SONT DECLAR ES HORS TRANSLATION-----end loop;-----end-MARK_DONT_MOVE_PAGES_WITHES;-----
0030 end-loop;-----end-MARK_TRANS_WITH;-----end-MARK_DONT_MOVE_PAGES;-----
procedure-----WRITE_UNIT--( COMP_UNIT_ARG : TREE )-----is-----package SEQ_IO is new SEQUENTIAL_IO( SECTOR );-----COMP_UNIT:= TREE;-----LIB_FILE:= SEQ_IO
.FILE_TYPE;-----function-----RECOPI E_POUR_COMPACTION--( T : TREE )-----return-----TREE-----is-----begin
-----UN SOURCE_POSITION N EST PAS A RECOPI ER-EN LUI MEME MAIS EXIGE DE RECOPI ER LE SOURCE LINE-ASSOCI -----if T.PT = S-then-----begin
-----return MAKE_SOURCE_POSITION(-----RECONSTRUIRE UNE POSITION-SOURCE-----RECOPI E_POUR_COMPACTION( GET_SOURCE_LINE( T ) ), GET
SOURCE_POSITION( T ) );-----AVEC UN DN_SOURCELINE TRANSL TE-----TOUTES LES  L MENTS TERMINAUX (PAS D ATTRIBUT PAS DE REF DE PAGE) SONT RENDUS TELS QU
ELS-----else if T.PT = HI or else T.PG = 0 then-----NUM_VAL COURTE OU NIL OU NON INITIALIS E AU TERMINAL STYLE DN_UNIVERSAL_INTEGER
return T;-----JUSTE-RETOURNER-L ARGUMENT-----UN SYMBOL_REP N EST-PAS A RECOPI ER EN LUI MEME MAIS EXIGE DE RECOPI ER-LE TXT_REP ASSOC
I -----else if T.TY = DN_SYMBOL_REP then-----POINTEUR A DN_SYMBOL_REP-----return RECOPI E_POUR_COMPACTION( D( XD_TEXT , T ) );-----
0040 | TRANSL TER LE TEXTE (UN TXTREP REMPLACE UN SYMREP ( ! ))-----UN POINTEUR SUR PAGE A NE PAS-TOUCHER EST A RETOURNER TEL QUEL-----else if DONT
MOVE( T.PG ) then-----PAGE A NE PAS TRANSL TER-----return T;-----NE RIEN FAIRE-----UN POINTEUR P/L FAISANT L OBJET DE REC
OPI E_POUR_COMPACTION-----else-----declare-----WORD_ZERO:= TREE:= DABS( 0, T );-----ENT TE DU NOEUD-POINTE PAR T-----begin-----
CAS OU L ENT TE DU NOEUD DONNE POUR RECOPI E EST DENATURE E EN POINTEUR-L CE QUI INDIQUE UNE RECOPI E D J FAITE-DE CE NOEUD-----if WORD_ZERO.PT = L
then-----ENT TE DE NOEUD-DEJ  RECOPI E CONTENANT UN POINTEUR L PG|LN AU REMPLACANT-----return ( P, TY=>WORD_ZERO.TY, PG=>WORD_ZERO.PG, LN=>
WORD_ZERO.LN );-----RETOURNER LE POINTEUR P RECONSTRUIT-AU REMPLAANT-----CAS OU L ENT TE DU NOEUD POINTE DONNE POUR RECOPI E EST NORMALE (HI VA
LU NSIZ)-----else-----CAS NORMAL D UNE ENT TE DE NOEUD-----case T.TY is-----QUELQUES AJUSTEMENTS SUIVANT TYPE DE NOEUD-----when C
LASS_NON_TASK_NAME | DN_TASK_SPEC =>-----D( XD_STUB, T, TREE_VOID );-----D( XD_BODY, T, TREE_VOID );-----when DN_INCOMPLETE =>-----D( XD_FULL_TYPE_S
PEC, T, TREE_VOID );-----when others =>-----null;-----end case-----RECOPI E_ET_MARQUAGE:-----declare-----LENGTH:= WO
RD_ZERO.NSIZ;-----COPI E_T:= TREE:= MAKE( T.TY, LENGTH );-----LPTR_TO_COPI E_T:= TREE:= ( L, TY=>COPI E_T.TY, PG=>COPI E_T.PG, LN=>COPI E_T.L
N );-----INITIALIS E:-----TREE;-----begin-----if T.TY = DN_NUM_VAL then-----POUR UNE VALEUR-NUM RIQUE-----DABS( 0, COPI E_T, DABS( 0, T ) );-----
0050 -----NE PAS OUBLIER L ENT TE QUI CONTIENT LE SIGNE-DANS ABSS-!!-----end if;-----DABS( 0, T, LPTR_TO_COPI E_T );-----ENSUITE REMPLACE L'E
NT TE-DU RECOPI E PAR UN POINTEUR L AU TRANSL TE INDIQUE-QUE LA TRANSLATION A EU LIEU-----if T.TY = DN_TXTREP or T.TY = DN_NUM_VAL then-----T
EXTE-OU VALEUR-NUM RIQUE-----for I in 1 .. LENGTH loop-----RECOPI ER LE CONTENU-----DABS( I, COPI E_T, DABS( I, T ) );-----end-loop
-----TOUTS AUTRES NOEUDS-----if T.TY = DN_COMPILATION_UNIT then-----POUR UNE UNIT  DE COMPACTION-----INITULE := RE
COPI E_POUR_COMPACTION( D( XD_LIB_NAME, T ) );-----TRANSL TER LE NOM EN LE CONVERTISSANT EN TXTREP (CONDUIT A DEUX RECOPI ES AVEC CI-DESSOUS ! )-----
end-if;-----PROPAGER RECOPI E:-----for I in 1 .. LENGTH loop-----POUR LES ATTRIBUTS-----DABS( I, COPI E_T, RECO
PI E_POUR_COMPACTION( DABS( I, T ) ) );-----METTRE LES ATTRIBUTS TRANSL TES-----end-loop-PROPAGER RECOPI E;-----end-if;-----
return COPI E_T;-----end-RECOPI E_ET_MARQUAGE;-----end-if;-----end;-----end-RECOPI E_POUR_COMPACTION;-----
-----begin-----COMP_UNIT := RECOPI E_POUR_COMPACTION ( COMP_UNIT_ARG );-----FAIRE-UNE RECOPI E COMPACT E-----CREER-LE-FICHER-UNIT :-----
declare-----SYM:= constant-----TREE:= D( XD_LIB_NAME, COMP_UNIT );-----PRENDRE LE SYMBOLE DU NOM-DE L'UNIT  DANS LA LIBRAIRIE-----FILE_NAME:= const
ant-----STRING:= IDL.LIB_PATH( 1 .. LIB_PATH_LENGTH ) & PRINT_NAME( SYM );-----CHA NE DU NOM PREFIX E-----begin-----SEQ_IO.CREATE( LIB_FILE, SEQ_IO.OUT
_FILE, FILE_NAME );-----CREER-LE-FICHER-LIBRAIRIE-----end-RECOPI E_ET_MARQUAGE;-----PREPARER- CRITURE-PAGES_U
NITE:-----declare-----FIRST_PAGE:= VPG_IDX:= COMP_UNIT.PG;-----PREMI RE PAGE DE LA RECOPI E COMPACT E-----NBR_PAGES:= NATURAL:= NATURAL( L
AST_BLOCK - FIRST_PAGE + 1 );-----NOMBRE DE PAGES-DE LA RECOPI E COMPACT E-----POINTEUR:= TREE:= ( P, TY=>DN_VOID, PG=>FIRST_PAGE, LN=>0 );-----
FABRIQU ER UN POINTEUR NON-TYPE POUR-TOUCHER LES PAGES-----INITULE:= TREE;-----begin-----DI( XD_NBR_PAGES, COMP_UNIT, NBR_PAGES );-----PO
INTER LE NOMBRE DE PAGES-DANS LE NOEUD RACINE (DN_COMPILATION_UNIT) DE LA RECOPI E----- CRIRE-LES-PAGES_D_UNIT :-----DE
S OPERATIONS D' CRITURE-A CONFINER DANS LE PAGE_MANAGER-----for I in 1 .. NBR_PAGES loop-----STOCKER LES PAGES-----INITULE := DABS( 0,
POINTEUR );-----TOUCHER LA PAGE-POUR LA FORCER DANS-UNE PAGE PHYSIQUE-----SEQ_IO.WRITE( LIB_FILE, PAG ASSOC PAGE( POINTEUR.PG ) ).DATA.all
);----- CRIRE LA PAGE-----POINTEUR.PG := POINTEUR.PG + 1;-----PAGE SUIVANTE-----end-loop- CRIRE-LES-PAGES_D_UNIT ;-----
end-RECOPI E_ET_MARQUAGE;-----end-FAIRE-UNE RECOPI E COMPACT E;-----SEQ_IO.CLOSE( LIB_FILE );-----NEW_UNIT_SEQ := APPEND( NEW_UNIT_SE
Q, COMP_UNIT );-----AJOUTER A LA LISTE DES UNIT S-----end-WRITE_UNIT;-----begin-----OPEN_IDL_TREE_FILE( IDL.LIB_PATH( 1 .. LIB_PATH_LENGTH
) & "###.TMP" );-----if DI( XD_ERR_COUNT, TREE_ROOT ) /= 0 then-----CLOSE_IDL_TREE_FILE;-----return;-----end-if;-----NEW_UNIT_SEQ := ( TREE_NIL, TREE_NIL
);-----declare-----USER_ROOT:= TREE:= D( XD_USER_ROOT, TREE_ROOT );-----RETIRER LA RACINE UTILISATEUR-----COMPACTION:= TREE:= D( XD_STRUCTURE,
USER_ROOT );-----EN EXTRAIRE LA COMPACTION-----COMP_UNIT_SEQ:= SEQ_TYPE:= LIST( D( AS_COMPLTN_UNIT_S, COMPACTION ) );-----PUIS LA LISTE D'UNI
TES DE-COMPACTION-----COMP_UNIT:= TREE;-----begin-----TRAITE-LES-UNIT S-DE-COMPACTION-----while not IS_EMPTY(
COMP_UNIT_SEQ ) loop-----POP( COMP_UNIT_SEQ, COMP_UNIT );-----if D( AS_ALL_DECL, COMP_UNIT ).TY = DN_VOID then-----UNIT -A PRAGMAS-SEULEM
ENT-----NEW_UNIT_SEQ := APPEND( NEW_UNIT_SEQ, COMP_UNIT );-----JUSTE-LA RECH NER DANS LA LISTE MISE A JOUR-----else-----UNIT -U
SUELLE-----MARK_DONT_MOVE_PAGES( COMP_UNIT );-----NEW_BLOCK;-----FORCE-A SE METTRE AU DEBUT D'UN NOUVEAU BLOC POUR ALIGNER LA RECOPI 
E COMPACT E-----WRITE_UNIT( COMP_UNIT );-----COMPACTER ET  CRIRE L UNIT  DE COMPACTION-----end-if;-----end-loop-TRAITE-LES-UNIT S-DE-C
OMPACTION;-----LIST( D( AS_COMPLTN_UNIT_S, COMPACTION ), NEW_UNIT_SEQ );-----REEMPLACER LA LISTE DES UNIT S
0080 PAR CELLE DES COMPACT ES, -SERA  CRIT A LA FERMETURE DU FICHER ARBRE-----end;-----DELETE_IDL_TREE_FILE;-----LE FICHER ###.TMP EST ABIM  PAR LE
PROCESSUS-DE MARQUAGE-----end-WRITE_LIB;-----
-----1-2-3-4-5-6-7-8-9-A-B-C-----
```